



Advanced OOPS & Design Patterns

Object-Oriented Thinking & Scalable System Design

Java 8+



Session Structure

1. Advanced OOPS principles
2. SOLID principles
3. Design thinking in Java
4. Creational patterns
5. Structural patterns
6. Behavioral patterns
7. Real-world architecture examples



Advanced OOPS Recap

Core pillars:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

Advanced focus:

- Composition over inheritance
- Loose coupling
- High cohesion



Encapsulation (Advanced View)

Goal:

Hide internal complexity.

```
class BankAccount {  
    private double balance;  
  
    public void deposit(double amount) {  
        if (amount > 0)  
            balance += amount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

Protect state integrity.



Abstraction

Expose behavior, hide implementation.

```
interface Payment {  
    void pay(double amount);  
}
```



Polymorphism

```
class CreditCard implements Payment {  
    public void pay(double amount) {  
        System.out.println("Paid via Card");  
    }  
}
```

Runtime binding.



Composition Over Inheritance

Bad:

Car extends Engine.

Good:

Car has Engine.

```
class Engine {}  
  
class Car {  
    private Engine engine;  
}
```

Better flexibility.



SOLID Principles

S → Single Responsibility

O → Open/Closed

L → Liskov Substitution

I → Interface Segregation

D → Dependency Inversion



Single Responsibility

One class → One reason to change.

Bad:

User handles DB + UI.

Good:

Separate service classes.



Open/Closed Principle

Open for extension

Closed for modification.

Use:

Interfaces + polymorphism.



Liskov Substitution

Subclass should replace parent safely.

Bad:

Bird → Penguin → fly() throws error.

Interface Segregation

Small, specific interfaces.

Avoid:

```
interface Machine {  
    void print();  
    void scan();  
    void fax();  
}
```

Better:

Separate interfaces.



Dependency Inversion

Depend on abstractions, not concretions.

```
class OrderService {  
    private Payment payment;  
  
    OrderService(Payment payment) {  
        this.payment = payment;  
    }  
}
```



Design Thinking Approach

1. Identify entities
2. Define responsibilities
3. Define relationships
4. Use interfaces
5. Apply patterns



Creational Patterns

Focus on object creation.

- Singleton
- Factory
- Builder

Singleton Pattern

```
class Singleton {  
    private static final Singleton INSTANCE =  
        new Singleton();  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        return INSTANCE;  
    }  
}
```

Used in:

Logger, Config manager.

Factory Pattern

```
interface Shape {  
    void draw();  
}  
  
class Circle implements Shape {  
    public void draw() {  
        System.out.println("Circle");  
    }  
}  
  
class ShapeFactory {  
    static Shape getShape(String type) {  
        if ("circle".equals(type))  
            return new Circle();  
        return null;  
    }  
}
```

Decouples object creation.



Builder Pattern

Used for complex objects.

```
class User {  
    private String name;  
    private int age;  
  
    private User(Builder builder) {  
        this.name = builder.name;  
        this.age = builder.age;  
    }  
  
    static class Builder {  
        private String name;  
        private int age;  
  
        Builder setName(String name) {  
            this.name = name;  
            return this;  
        }  
  
        Builder setAge(int age) {  
            this.age = age;  
            return this;  
        }  
  
        User build() {  
            return new User(this);  
        }  
    }  
}
```



Structural Patterns

Focus on object relationships.

- Adapter
- Decorator
- Proxy

THIRAVUGAL



Adapter Pattern

Convert incompatible interface.

```
class OldSystem {  
    void specificRequest() {}  
}  
  
class Adapter {  
    private OldSystem oldSystem;  
  
    Adapter(OldSystem oldSystem) {  
        this.oldSystem = oldSystem;  
    }  
  
    void request() {  
        oldSystem.specificRequest();  
    }  
}
```



Decorator Pattern

Add behavior dynamically.

```
interface Coffee {  
    double cost();  
}  
  
class SimpleCoffee implements Coffee {  
    public double cost() { return 5; }  
}  
  
class MilkDecorator implements Coffee {  
    private Coffee coffee;  
  
    MilkDecorator(Coffee coffee) {  
        this.coffee = coffee;  
    }  
  
    public double cost() {  
        return coffee.cost() + 2;  
    }  
}
```



Behavioral Patterns

Focus on communication.

- Observer
- Strategy
- Command

THIRAVUGAL

Strategy Pattern

```
interface DiscountStrategy {  
    double apply(double price);  
}  
  
class SeasonalDiscount implements DiscountStrategy {  
    public double apply(double price) {  
        return price * 0.9;  
    }  
}
```

Switch behavior dynamically.



Observer Pattern

```
interface Observer {  
    void update(String message);  
}
```

Used in:

Event systems, UI frameworks.



Real World Architecture Example

E-commerce system:

- Strategy → Discount
- Factory → Payment methods
- Singleton → Configuration
- Observer → Order updates

Performance Considerations

Good design:

- Reduces tight coupling
- Improves maintainability
- Enables scalability

Poor design:

- Increases complexity
- Hard to test
- Difficult to extend



Common Mistakes

- Overusing inheritance
- Not following SOLID
- Creating god classes
- Tight coupling
- Ignoring design patterns



Interview Discussion Points

- Why composition over inheritance?
- When to use Singleton?
- Difference between Factory & Builder?
- Real example of Strategy pattern?
- How to design scalable systems?



Summary

Advanced OOPS builds:

- Maintainable systems
- Scalable architecture
- Clean code structure
- Reusable components

Master:

- SOLID principles
- Composition
- Core design patterns
- Real-world application thinking